Field Hours

CSCE 221 Data Structures and Algorithms

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 50 minutes

Cost, arrays, and lists

A data structure is a way of storing values plus the cost of each operation. State the cost in the number of elements, n, before you write the code. An array stores items in contiguous memory. Reading the item at index i is one step. Inserting at the front shifts every later item, so it costs n.

A linked list stores each item with a pointer to the next. Inserting at the front is one step, because you do not move the others. Finding the item at index i walks i pointers, so it costs n in the worst case. The structure you pick is the operation you do most often.

The same five items, two layouts.

Array

Index 2 is one read

Insert at the front moves four items

Linked list

Front insert is one pointer

Index 2 walks two pointers

Insert 9 at the front of [2, 4, 6, 8]

1. Array: shift 8, 6, 4, then 2 one slot right. Four moves. Then write 9 at index 0.

2. List: a new node holding 9 points at the old front. One pointer write.

Problem 1. You look up by index a million times and insert at the front ten times. Which structure?

Answer. The array. The million index reads are cheap. The ten front inserts are the expensive operation, and they are rare.

Problem 2. You insert at the front a million times and almost never read by index. Which structure?

Answer. The linked list. Each front insert is one step. An array would shift the whole collection every time.

Lecture 2 · 50 minutes

Stacks and queues

A stack is last-in, first-out. Push and pop touch only the top, so both are one step if the top is at the end of an

array or the front of a list. A queue is first-in, first-out. If you implement it with a single array and remove from the front, each remove shifts everyone. A circular array, or a list with a tail pointer, makes enqueue and dequeue one step.

Call stacks, undo, and depth-first search are stacks. Printers, buffers, and breadth-first search are queues. Name the order. The order is the data structure.

A queue with a head and a tail.

1. Enqueue writes at the tail

2. Dequeue reads at the head

3. Neither walks the middle

Six operations

1. Start empty. Enqueue A, enqueue B, dequeue, enqueue C, dequeue, dequeue.

2. After the operations the results are A, then B, then C. A stack with the same push and pop order would have returned B before A.

Problem 1. Breadth-first search uses which structure, and what goes wrong if you use the other?

Answer. A queue. A stack makes it depth-first: you finish one path before the neighbors. The first time you reach a node is then not a shortest path in hops.

Problem 2. Why is removing from the front of a plain array not a one-step dequeue?

Answer. Every later item moves one slot. That is n moves. A head index, or a tail pointer on a list, avoids the shift.

Lecture 3 · 50 minutes

Trees and binary search

A binary search tree stores a key in each node. Smaller keys go left, larger keys go right. If the tree stays balanced, the height is about log base 2 of n, and search, insert, and delete follow one path of that length. If you insert already-sorted keys into a plain tree, it becomes a list and every operation costs n.

Balance is the whole point of later trees, AVL and red-black. This course only requires you to see the failure: sorted input, no rebalance, linear cost. Say the height before you quote log n.

A balanced tree of seven keys.

Root 4

Left 2, right 6

Leaves 1, 3, 5, 7

Search for 5

1. Start at 4. 5 is larger, go right to 6.

2. 5 is smaller than 6, go left to 5. Three comparisons. The height is 2.

3. The same keys inserted in order 1, 2, 3, 4, 5, 6, 7 make a spine. Searching for 7 walks seven nodes.

Problem 1. You insert a million sorted keys into a tree that does not rebalance. What is the cost of the last search?

Answer. About a million steps. The tree is a linked list leaning right. Log n applies only when the height is logarithmic.

Problem 2. A balanced tree has height 20. About how many keys can it hold?

Answer. On the order of 2^20, about a million. Each extra level doubles the capacity.

Lecture 4 · 50 minutes

Hash tables

A hash table stores a key in a bucket chosen by a hash function. With chaining, each bucket is a list. If the hash spreads n keys across n buckets, the average list is one key long and search is expected constant time. The expectation is over the hash, not a promise about one unlucky function.

A load factor above 1 means the lists are growing. Resize and rehash before you blame the hash. A hash of the object's address changes if the object moves. Hash the value.

Four buckets, chaining.

Bucket 0: empty

Bucket 1: one key

Bucket 2: two keys that collided

Bucket 3: one key

Eight keys, four buckets

1. The load factor is 8/4 = 2. Average chain length is 2 if the hash is even.

2. One search walks about two nodes, not eight.

3. Grow to sixteen buckets and rehash. The load drops to 1/2. That resize costs a full pass, once, not on every lookup.

Problem 1. All eight keys land in bucket 0. What failed?

Answer. The hash, or an adversary who knew it. Chaining still works, but search is a list of eight. A usable hash spreads keys. Universal hashing is the graduate version of this warning.

Problem 2. Why do you rehash into a larger table instead of only lengthening the chains?

Answer. Long chains make every lookup linear. A larger table, with the same hash family, brings the average chain back down. The resize is the expensive step you pay rarely.

Lecture 5 · 50 minutes

Heaps

A binary heap is an array that pretends to be a complete tree. The parent of index i is at floor((i - 1) / 2). The smallest key, in a min-heap, sits at index 0. Insert places the new key at the end and sifts it up. Remove-min swaps the root with the last key, drops the last, and sifts down. Both are proportional to the height, which is log n.

Use a heap when you repeatedly need the next smallest item and you do not need the full sorted order. That is Dijkstra's frontier, and it is why a heap beats a sorted array for those updates.

A min-heap in array order.

Index 0 holds 1

Indexes 1 and 2 hold 4 and 3

Each parent is smaller than its children

Insert 2 into [1, 4, 3]

1. Append 2 at index 3. Parent of 3 is floor(2/2) = 1, which holds 4.

2. 2 is smaller than 4, so they swap. The array is [1, 2, 3, 4].

3. Parent of index 1 is 0, which holds 1. 2 is larger, so the sift stops. Two comparisons.

Problem 1. Remove-min on a heap of n items costs what, and what do you get back?

Answer. O(log n) time, and the smallest key. The new root is the old last key sifted down until it is smaller than both children, or it is a leaf.

Problem 2. Why not sort the array once and then read it in order, if you need every item eventually?

Answer. You can, if no new item arrives. A heap is for a set that changes. Sorting again after each insert would cost n log n per insert. The heap costs log n.

Lecture 6 · 50 minutes

Graphs you can search

A graph is vertices and edges. An adjacency list stores, for each vertex, the vertices it touches. Space is proportional to vertices plus edges. Breadth-first search from a source uses a queue and visits each vertex and edge a constant number of times, so the cost is vertices plus edges. On an unweighted graph, the first visit is a shortest path in hops.

Depth-first search uses a stack, or recursion, and answers a different question: what is reachable, and where are the cycles. Do not quote a shortest path from a depth-first order.

Breadth-first search.

1. Queue starts with the source

2. Take the front vertex

3. Enqueue its unvisited neighbors

4. The first visit is fewest hops

A line of four vertices

1. Vertices A-B-C-D, edges between neighbors. Start at A.

2. Queue: A. Visit A, enqueue B. Visit B, enqueue C. Visit C, enqueue D. Visit D.

3. Distance from A is 0, 1, 2, 3. Depth-first from A might still visit them in that order on a line, but a branch would not. The queue is what makes the distances right.

Problem 1. An adjacency matrix uses n by n entries. When is the list the better choice?

Answer. When the graph is sparse, edges much fewer than n squared. The matrix spends space on the missing edges. The list spends space only on the edges that exist. Scanning all neighbors of one vertex is also faster on the list when the degree is small.

Problem 2. BFS reports a distance of 5. The graph has a weighted edge of cost 1 that BFS ignored because it only counts hops. Is 5 still the cheapest path?

Answer. Not necessarily. BFS shortest paths are in hops, or in equal edge weights. Different weights need Dijkstra, which is the graduate graph lecture, and a heap from the previous one.